



Trabajo Práctico — Programación Funcional

Bot de Arbitraje Triangular en Haskell

Arbitrín

Interfaz con la API de Binance Testnet

Bendayan, Alberto — Legajo: 62786

Boullosa Gutiérrez, Juan Cruz — Legajo: 63414

8 de mayo de 2026

Índice

1. Introducción	4
2. Descripción del problema	5
2.1. Arbitraje triangular	5
2.2. Comisiones y su efecto sobre la rentabilidad	5
2.3. Precios bid y ask	6
2.4. Espacio de caminos posibles	7
2.5. Restricciones del sistema real	7
3. Arquitectura del sistema	9
3.1. Flujo de ejecución	10
3.2. Capa de entrada: integración con Binance	11
3.3. Capa de transformación: núcleo puro	12
3.4. Capa de orquestación: monad stack	12
3.5. Capa de notificación	13
3.6. Consulta periódica y comandos de Telegram	13
4. Modelo de dominio	19
4.1. Tipos primitivos	19
4.2. Snapshot de mercado	19
4.3. Camino triangular y oportunidad	20
4.4. Plan de ejecución y resultado	20
5. Principios de programación funcional aplicados	22
5.1. Pureza e inmutabilidad en el núcleo de arbitraje	22
5.2. Tipos como documentación: newtypes y ADTs	22
5.3. Abstracción con typeclasses	23
5.4. Manejo de efectos con monad transformers	23
5.5. Manejo de errores sin excepciones	25
5.6. Representación de texto con <code>Data.Text</code>	25
6. Implementación destacada	27
6.1. Loop continuo como recursión en <code>BotM</code>	27
6.2. Simulación de un camino con la mónada <code>Maybe</code>	28
6.3. Detección de oportunidades con list comprehensions	29
6.4. Ejecución de una ronda en <code>BotM</code>	30

7. Resultados	32
7.1. Corrida sobre Binance Testnet	32
7.2. Corrida con exchange simulado	32
7.3. Loop continuo y comandos de Telegram	34
8. Conclusiones	36
8.1. Aprendizajes	36
8.2. Limitaciones	36
8.3. Posibles extensiones	37

1. Introducción

Este trabajo práctico consiste en el diseño e implementación de un bot de **arbitraje triangular** sobre criptomonedas, utilizando la API pública de *Binance Testnet* como fuente de datos de mercado y plataforma de ejecución de órdenes. Al finalizar cada ronda de operaciones, el sistema registra el resultado (órdenes ejecutadas, Profit and Loss (PnL) obtenido y eventuales errores) y envía una notificación con ese detalle a través de un bot de *Telegram*. El bot corre en un ciclo continuo con un intervalo configurable entre rondas, y responde a comandos de consulta (`/balance`, `/status`, `/pnl`, `/open_orders`, `/history [N]`) enviados directamente por Telegram.

El arbitraje triangular es una estrategia de trading que busca aprovechar incoherencias de precio entre tres pares de activos relacionados. Si los tipos de cambio entre los pares A/B, B/C y C/A no son perfectamente consistentes entre sí, es posible intercambiar un activo inicial a través de esa cadena de tres operaciones y recuperar más del activo original del que se comenzó. Este trabajo implementa la detección automática de esas oportunidades y, cuando el margen supera un umbral configurable, la ejecución de las órdenes correspondientes.

Un aspecto central del modelo es el tratamiento explícito de las **comisiones de trading**: cada operación dentro de la cadena triangular tiene un costo asociado que reduce el monto efectivamente recibido. El bot obtiene las tasas de comisión reales directamente desde la API de Binance y las incorpora en la simulación de cada camino antes de tomar cualquier decisión de ejecución, garantizando que sólo se opere cuando la ganancia estimada supere los costos totales de las tres órdenes.

El sistema fue desarrollado en **Haskell**, y su estructura refleja los principios del paradigma funcional: un núcleo de lógica de negocio completamente puro, sin efectos secundarios, que opera sobre tipos algebraicos bien definidos, rodeado por capas que administran la interacción con el mundo exterior (llamadas HTTP, lectura de configuración, estado mutable, notificaciones y persistencia) de forma explícita y controlada mediante *monad transformers*.

2. Descripción del problema

2.1. Arbitraje triangular

El arbitraje triangular explota inconsistencias en los tipos de cambio entre tres activos. Dados los activos A , B y C , y los precios de mercado P_1 (unidades de A por unidad de B), P_2 (unidades de B por unidad de C) y P_3 (unidades de A por unidad de C), el **factor de retorno bruto** de la cadena $A \rightarrow B \rightarrow C \rightarrow A$ es:

$$R = \frac{P_3}{P_1 \cdot P_2} \quad (1)$$

Si $R > 1$, la cadena genera una ganancia; si $R < 1$, produce una pérdida. En un mercado perfectamente eficiente se cumple $R = 1$ en todo momento; las oportunidades surgen de desequilibrios transitorios entre los distintos pares de trading.

Ejemplo numérico. Consideremos los activos USDT, BTC y ETH con los siguientes precios de mercado:

Par	Precio	Interpretación
BTC/USDT	50 000	1 BTC cuesta 50 000 USDT
ETH/BTC	0,05	1 ETH cuesta 0,05 BTC
ETH/USDT	2 600	1 ETH cuesta 2 600 USDT

Partiendo de 1 000 USDT, la cadena opera así:

1. **USDT $\rightarrow$ BTC:** $\frac{1\,000}{50\,000} = 0,02$ BTC
2. **BTC $\rightarrow$ ETH:** $\frac{0,02}{0,05} = 0,4$ ETH
3. **ETH $\rightarrow$ USDT:** $0,4 \times 2\,600 = 1\,040$ USDT

Aplicando la fórmula (1): $R = \frac{2600}{50000 \times 0,05} = 1,04$, lo que representa una ganancia bruta del 4 % sobre el capital inicial.

2.2. Comisiones y su efecto sobre la rentabilidad

Cada orden ejecutada en un exchange cobra una comisión sobre el monto recibido. Si la tasa de comisión es c , el monto efectivo tras cada paso es un factor $(1 - c)$ menor. Aplicado a las tres operaciones de la cadena, el **factor de retorno neto** resulta:

$$R_{\text{neto}} = \frac{P_3}{P_1 \cdot P_2} \cdot (1 - c)^3 \quad (2)$$

La condición de rentabilidad es simplemente $R_{\text{neto}} > 1$.

Ejemplo (continuación). Con una comisión de $c = 0,1\%$ por operación:

1. $0,02 \times (1 - 0,001) = 0,01998$ BTC
2. $\frac{0,01998}{0,05} \times (1 - 0,001) = 0,39920$ ETH
3. $0,39920 \times 2\,600 \times (1 - 0,001) \approx 1\,036,88$ USDT

La ganancia neta es de 36,88 USDT ($\approx 3,69\%$), sensiblemente menor que el 4 % bruto. En escenarios donde el retorno bruto es apenas superior a 1, las comisiones pueden eliminar completamente la ganancia. Por este motivo el bot consulta las tasas reales de comisión a la API de Binance y las aplica en la simulación de cada camino antes de tomar una decisión.

Existe además una sutileza en el cómputo del PnL *real* tras la ejecución: las comisiones de cada paso se cobran en el activo recibido en ese paso, no necesariamente en USDT. En la cadena $\text{USDT} \rightarrow \text{BTC} \rightarrow \text{ETH} \rightarrow \text{USDT}$, la primera orden descuenta comisión en BTC y la segunda en ETH. Un cálculo que solo compare el monto final de USDT con el inicial ignora estas comisiones intermedias. El sistema calcula el PnL neto sumando los deltas de todos los activos (en BTC, ETH y USDT) convertidos a USDT usando cotizaciones bid/ask del snapshot (bid para deltas positivos, ask para negativos), obteniendo así un resultado que refleja el costo total real de la operación.

2.3. Precios bid y ask

Un libro de órdenes tiene siempre dos precios para un par dado: el **bid** (el mayor precio que un comprador está dispuesto a pagar por el activo base) y el **ask** (el menor precio al que un vendedor está dispuesto a ceder el activo base). La diferencia entre ambos se llama *spread*.

Cada paso de la cadena triangular convierte un activo A en un activo B . El precio a utilizar depende de cómo el exchange lista el par correspondiente:

- **Venta de base (par directo).** Si el exchange tiene el par A/B (donde A es el activo base), el bot *vende* A y recibe B al precio **bid**:

$$\text{cantidad_out} = \text{bidPrice} \times \text{cantidad_in} \times (1 - c)$$

- **Compra de base (par invertido).** Si el exchange solo lista el par B/A (donde B es el activo base), el bot *compra* B pagando con A al precio **ask**:

$$\text{cantidad_out} = \frac{\text{cantidad_in}}{\text{askPrice}} \times (1 - c)$$

Usar el precio incorrecto según la dirección sobreestimaría el retorno real y llevaría a ejecutar operaciones que en la práctica son deficitarias. Esta distinción es especialmente relevante en pares con spread amplio, donde la diferencia entre bid y ask puede por sí sola eliminar la ganancia esperada.

2.4. Espacio de caminos posibles

Con cuatro activos disponibles (BTC, ETH, BNB, USDT), la cantidad de combinaciones de tres activos distintos es $\binom{4}{3} = 4$, y para cada terna existen dos direcciones de recorrido. El espacio total de caminos triangulares es por lo tanto de $4 \times 2 = 8$ caminos, enumerados en la Tabla 1.

#	Paso 1	Paso 2	Paso 3
1	BTC/ETH	ETH/BNB	BNB/BTC
2	BTC/BNB	BNB/ETH	ETH/BTC
3	BTC/ETH	ETH/USDT	USDT/BTC
4	BTC/USDT	USDT/ETH	ETH/BTC
5	BTC/BNB	BNB/USDT	USDT/BTC
6	BTC/USDT	USDT/BNB	BNB/BTC
7	ETH/BNB	BNB/USDT	USDT/ETH
8	ETH/USDT	USDT/BNB	BNB/ETH

Cuadro 1: Todos los caminos triangulares posibles con los 4 activos soportados.

El tamaño reducido del espacio de búsqueda permite que el bot evalúe los 8 caminos en cada ciclo sin ningún tipo de poda o heurística: la enumeración exhaustiva es tanto correcta como eficiente para este universo de activos.

2.5. Restricciones del sistema real

La implementación opera bajo las siguientes restricciones:

- **Universo de activos.** El bot trabaja con cuatro activos: BTC, ETH, BNB y USDT. Dado que los caminos triangulares tienen longitud fija igual a 3, el espacio de caminos posibles es finito y pequeño, lo que permite enumerarlos y evaluarlos exhaustivamente en cada ciclo.
- **Binance Testnet.** Las operaciones se ejecutan sobre el entorno de pruebas de Binance (`testnet.binance.vision`), que replica la API de producción con datos de precio reales pero sin dinero real.
- **Órdenes de mercado.** El bot utiliza exclusivamente órdenes de mercado, que se ejecutan de forma inmediata al mejor precio disponible en el libro de órdenes. Esto garantiza la ejecución pero no el precio exacto simulado.

- **Capital máximo configurable.** El riesgo por operación está acotado por el parámetro `BOT_MAX_TRADE_AMOUNT_USDT`: para cada activo con balance, se calcula el equivalente de ese tope en la moneda del activo y se usa ese valor como cantidad máxima a operar. El arbitraje puede arrancar desde cualquier activo con balance positivo (BTC, ETH, BNB o USDT), siempre que exista una ruta triangular rentable desde ese punto de partida.

3. Arquitectura del sistema

El sistema está organizado en tres capas con responsabilidades claramente separadas. El criterio de separación es el **tipo de operación** que realiza cada módulo: interacción con el mundo exterior, lógica pura de negocio, u orquestación del flujo de ejecución.

Capa	Módulos	Responsabilidad
Entrada	<code>Binance.API.*</code> , <code>FakeExchange.*</code>	Integración con exchanges (real y simulado)
Transformación	<code>Bot.Domain</code> , <code>Bot.Arbitraje</code> , <code>Bot.Pricing</code> , <code>Exchange.Interface</code>	Lógica pura sin efectos
Orquestación	<code>Bot.Runtime</code> , <code>Bot.Config</code> , <code>Bot.Persist</code> , <code>Main</code>	Coordinación, estado y persistencia
Notificación	<code>Notification.Types</code> , <code>Notification.Telegram</code>	Envío de alertas proactivas
Comandos	<code>Notification.TelegramCommands</code>	Polling de comandos y respuesta interactiva

La Figura 1 muestra la relación entre las capas y el sentido del flujo de datos.

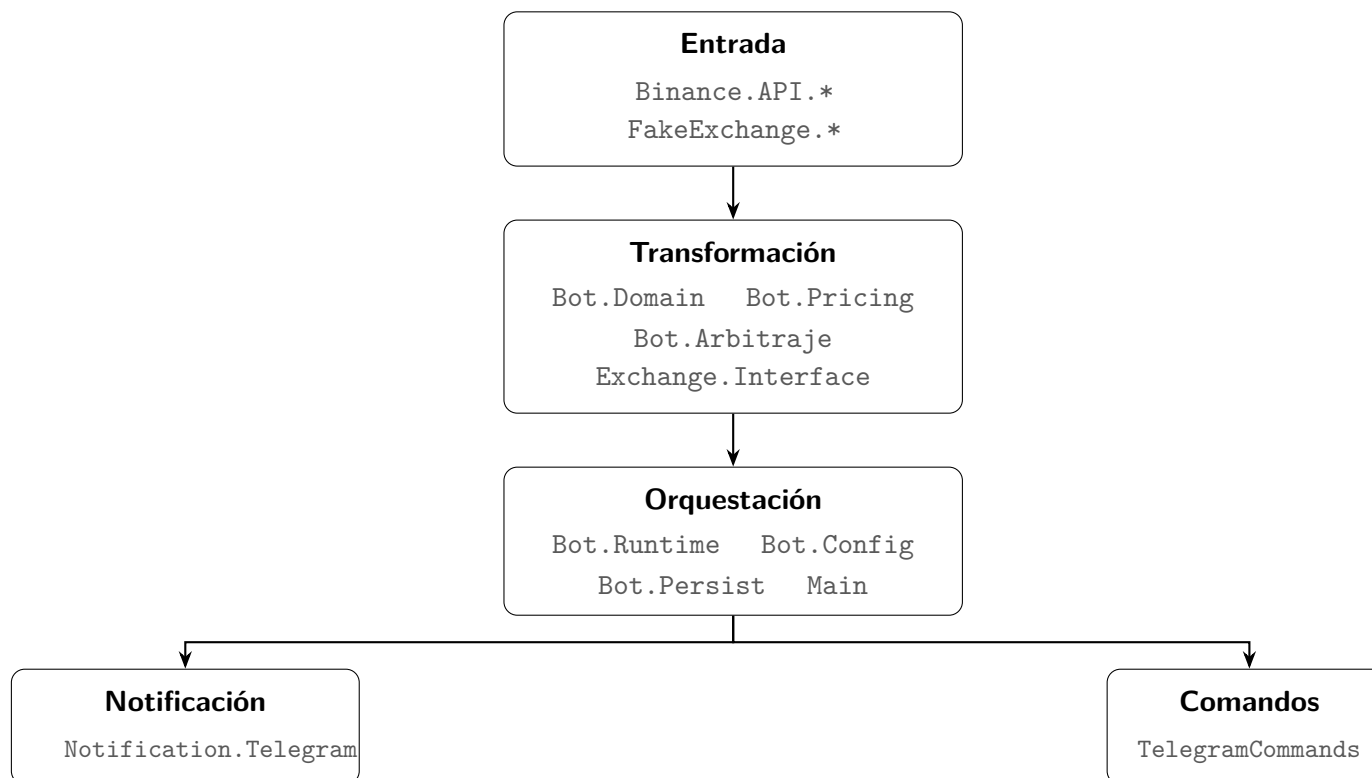


Figura 1: Diagrama de capas y módulos principales del sistema.

3.1. Flujo de ejecución

Al iniciarse, `Main` realiza una única inicialización y luego entra en un ciclo continuo. Los pasos de inicialización son:

1. **Carga de configuración.** `Bot.Config` lee las variables de entorno del archivo `.env`: credenciales de API, umbral mínimo de ganancia, monto máximo por ronda, intervalo de polling (`BOT_POLL_INTERVAL`, por defecto 30 segundos), parámetros de notificación y la ruta del archivo de estado persistido.
2. **Carga del estado persistido.** `Bot.Persist` intenta leer el archivo de estado (`bot_state.json` por defecto). Si existe, recupera el contador de rondas y el PnL acumulado de sesiones anteriores. Si no existe o está dañado, el bot arranca desde cero sin interrumpir la ejecución.
3. **Inicialización del estado compartido.** Se crea un `IORef BotState` que actúa como memoria compartida entre el hilo principal del bot y el hilo de comandos de Telegram.
4. **Lanzamiento del listener de comandos.** Si Telegram está habilitado, se lanza un hilo concurrente (`forkIO`) que ejecuta `runCommandListener`: un loop de long-polling sobre `getUpdates` que atiende comandos entrantes de forma independiente al ciclo principal.

A partir de ese punto, el bot entra en `botLoop`: una función recursiva en cola dentro de `BotM` que ejecuta indefinidamente los siguientes pasos por ronda:

1. **Verificación de conectividad.** `checkConnectivityOrThrow` realiza un `ping` al exchange. Si falla, lanza un `BotError` que es capturado por `catchError` en el loop, registrado en consola, y la ronda se omite sin detener el bot.
2. **Obtención del snapshot de mercado.** Se consultan los precios bid/ask (`getBookTicker`) y las tasas de comisión reales (`getTradeFees`) para todos los pares del universo de activos. Los datos se combinan en un `MarketSnapshot`.
3. **Generación de caminos y detección (puro).** `allTriangularPaths` enumera los 8 caminos posibles. `buildCandidateAmounts` determina, para cada activo con balance positivo, el monto a operar acotado al equivalente en USDT de `BOT_MAX_TRADE_AMOUNT_USDT` usando `Bot.Pricing`. `detectOpportunities` se ejecuta para cada monto candidato y los resultados se concatenan, de modo que el bot puede detectar oportunidades que partan de cualquier activo disponible en cartera, no exclusivamente de USDT.

4. **Decisión (puro).** `makeDecision` aplica el umbral mínimo de ganancia y selecciona la mejor oportunidad, devolviendo `DoTrade opp` o `NoTrade`.
5. **Ejecución (si aplica).** En caso de `DoTrade`, se construye el `ExecutionPlan` y se invoca `executeRound` dentro de `BotM`, que coloca las tres órdenes de mercado en secuencia y registra el resultado en `BotState`.
6. **Notificación y persistencia.** Se envía un mensaje a Telegram con el resultado de la ronda. `Bot.Persist` serializa el estado actualizado a disco. El `IORef BotState` se actualiza con el nuevo estado para que el listener de comandos lo pueda leer.
7. **Espera y siguiente ronda.** `threadDelay` pausa el hilo durante `BOT_POLL_INTERVAL` segundos antes de iniciar la siguiente ronda.

3.2. Capa de entrada: integración con Binance

Esta capa concentra todo el código que produce efectos de entrada/salida. Sus módulos se encargan de construir y firmar las peticiones HTTP, parsear las respuestas JSON y exponer los datos al resto del sistema a través de tipos propios del dominio.

- `Binance.API.Client`: funciones que ejecutan las llamadas HTTP: verificación de conectividad (`ping`), obtención de precios de compra/venta (`getBookTicker`), consulta de comisiones reales (`getTradeFees`) y colocación de órdenes de mercado (`placeMarketOrder`).
- `Binance.API.Auth`: firma HMAC-SHA256 de los parámetros de cada petición autenticada, requerida por la API de Binance.
- `Binance.API.Conversion`: convierte las respuestas crudas de la API (tipos de `Binance.API.Types`) a los tipos del dominio definidos en `Bot.Domain`.
- `Binance.API.Instance`: implementa la typeclass `Exchange` para Binance, actuando como punto de unión entre la capa de entrada y el resto del sistema.
- `FakeExchange.Instance` / `FakeExchange.Control`: implementación simulada del exchange que opera completamente en memoria, sin conexión a red. Mantiene un mapa de precios configurable y genera fills sintéticos. Permite ejecutar el bot en modo de prueba activando `BOT_EXCHANGE=fake` en el archivo `.env`, lo que facilita el desarrollo y la verificación del comportamiento sin consumir créditos de la testnet ni depender de conectividad.

3.3. Capa de transformación: núcleo puro

Esta capa contiene únicamente código puro: ninguna función produce efectos secundarios ni vive en ninguna mónada de efectos. Dado un estado del mercado, las funciones de esta capa calculan de forma determinista qué caminos son rentables y cuál es el mejor.

- **Bot.Domain:** define todos los tipos del dominio: `Asset`, `Pair`, `TriangularPath`, `MarketSnapshot`, `ArbOpportunity`, `ExecutionPlan` y `RoundResult`, entre otros. Se detalla en la Sección 4.
- **Bot.Arbitraje:** implementa la lógica de detección de oportunidades: enumera todos los caminos triangulares posibles (`allTriangularPaths`), simula cada uno contra el snapshot de mercado (`simulatePath`), filtra los rentables (`detectOpportunities`) y elige el mejor (`makeDecision`).
- **Bot.Pricing:** funciones puras de conversión a USDT. `assetUsdtRateWhenSelling` devuelve el precio bid del par `asset/USDT` (o el inverso del ask si el par listado es `USDT/asset`), y `assetUsdtRateWhenBuying` hace lo análogo con el ask. USDT se mapea directamente a 1. Ambas devuelven `Either Text Double`: si no existe cotización a USDT para el activo en el snapshot, retornan `Left` con un mensaje de error en lugar de lanzar una excepción.

La pureza de esta capa tiene una consecuencia directa: sus funciones son fácilmente testeables de forma unitaria sin necesidad de mocks ni de conexión a ningún servicio externo.

3.4. Capa de orquestación: monad stack

Esta capa coordina la ejecución completa de una ronda de trading, administrando de forma explícita las tres dimensiones de efecto que el sistema necesita:

- **Configuración de solo lectura:** las credenciales de API, umbrales de ganancia y límites de capital se inyectan a través de un `ReaderT Env`.
- **Estado mutable:** el historial de rondas ejecutadas se acumula en un `StateT BotState`.
- **Manejo de errores:** los fallos recuperables (error de red, orden rechazada) se propagan mediante `ExceptT BotError` sin interrumpir abruptamente el programa.

La combinación de estos tres transformadores constituye la mónada `BotM`, definida en `Bot.Runtime`, que se describe con más detalle en la Sección 5.

El módulo `Bot.Persist` se encarga de la persistencia entre sesiones. Define el tipo `PersistedState` (que contiene el contador de rondas y el PnL acumulado por activo) y expone funciones puras de conversión (`fromBotState`, `applyToInitialState`) junto con operaciones de IO para leer y escribir el archivo JSON (`loadState`, `saveState`).

El `BotState` completo contiene campos adicionales que no se persisten a disco pero son fundamentales para la operación en memoria: el historial reciente de rondas, las órdenes abiertas durante una ejecución, los balances delta acumulados desde fills, y `bsLastFetchedBalances`, que almacena los balances reales obtenidos del exchange en el último ciclo. Este último campo es el que consume el comando `/balance` de Telegram para responder con datos actualizados sin necesidad de una nueva llamada al exchange.

El módulo `Main` actúa como punto de entrada: carga la configuración y el estado persistido, construye el entorno, invoca las capas de entrada y transformación en secuencia, delega la ejecución de órdenes a `Bot.Runtime`, y coordina el guardado del estado y el envío de notificaciones al finalizar cada ciclo.

3.5. Capa de notificación

Esta capa agrupa los módulos responsables de comunicar el resultado de cada ciclo al exterior del sistema.

- `Notification.Types`: define el tipo de error `TelegramError`, utilizado para representar fallos en el envío de mensajes (`TelegramSendError` y `TelegramConfigError`).
- `Notification.Telegram`: implementa el envío de mensajes al Bot API de Telegram vía HTTPS. Expone `sendTelegramMessage` y las funciones de formato `formatDecision`, `formatRoundResult` y `formatExecutionError`, que convierten los tipos del dominio en texto legible para el operador.

Los errores de notificación son tratados como no críticos: si el envío a Telegram falla, el bot los registra por consola pero continúa su ejecución normal.

3.6. Consulta periódica y comandos de Telegram

Además del envío proactivo de notificaciones, el bot expone cinco comandos de consulta que el operador puede invocar en cualquier momento enviando un mensaje al bot de Telegram:

- `/balance`: devuelve los balances reales obtenidos en el último ciclo de consulta al exchange, por activo (BTC, ETH, BNB, USDT).
- `/status`: devuelve el estado general del bot, número de rondas ejecutadas, resultado de la última ronda, cantidad de órdenes abiertas y uptime del proceso.
- `/pnl`: devuelve el PnL acumulado por activo desde el inicio de la sesión y el total de rondas ejecutadas.
- `/open_orders`: devuelve la cantidad de órdenes en curso. En condiciones normales este valor es cero salvo durante la ventana de ejecución de una ronda.
- `/history [N]`: devuelve las últimas N operaciones ejecutadas (por defecto 5, máximo 20). Por cada ronda muestra el estado (`RoundSuccess` / `RoundPartial` / `RoundFailed`), la ruta triangular recorrida, los montos de entrada y salida en USDT, y el PnL neto de esa ronda. Los datos provienen del campo `bsTradeHistory` del `BotState`, que acumula un registro `PersistedRound` por cada ronda completada.

La implementación utiliza el endpoint `getUpdates` de la API de Telegram con *long-polling*: el hilo de comandos realiza una petición con `timeout=25` segundos y procesa los updates recibidos antes de emitir la siguiente. Esto garantiza latencia baja sin consumir recursos en un busy-loop.

El acceso al estado es concurrente: el hilo principal escribe el `IORef BotState` al finalizar cada ronda, y el hilo de comandos lo lee cuando recibe un comando. Al ser un único escritor y un único lector, `IORef` es suficiente sin necesidad de `TVar` ni locks explícitos.

El comando `/history [N]` acepta un argumento numérico opcional que acota la cantidad de entradas mostradas; si se omite o es inválido, el valor por defecto es 5. El parámetro se parsea con `readMaybe` y se sujeta al rango `[1, 20]` antes de pasarse a `fmtHistory`, manteniendo la lógica de validación en el mismo sitio que el dispatch.

```

1  runCommandListener :: Config -> IORef BotState -> UTCTime
   -> IO ()
2  runCommandListener config stateRef startTime = go 0
3  where
4      go lastId = do
5          updates <- fetchUpdates config (lastId + 1)
6                  'catch' \( _ :: SomeException ) ->
7                      return []
8          mapM_ (handleUpdate config stateRef startTime)
                updates

```

```
8      let nextId = foldl (\acc u -> max acc (tgUpdateId u
      )) lastId updates
9      go nextId
```

Listing 1: Notification.TelegramCommands: loop del listener

El parámetro `lastId` actúa como cursor: al pasar `offset = lastId + 1` en cada petición, Telegram descarta los updates ya procesados y entrega solo los nuevos. El listener es también recursivo en cola, acumulando el cursor sin construir stack frames.

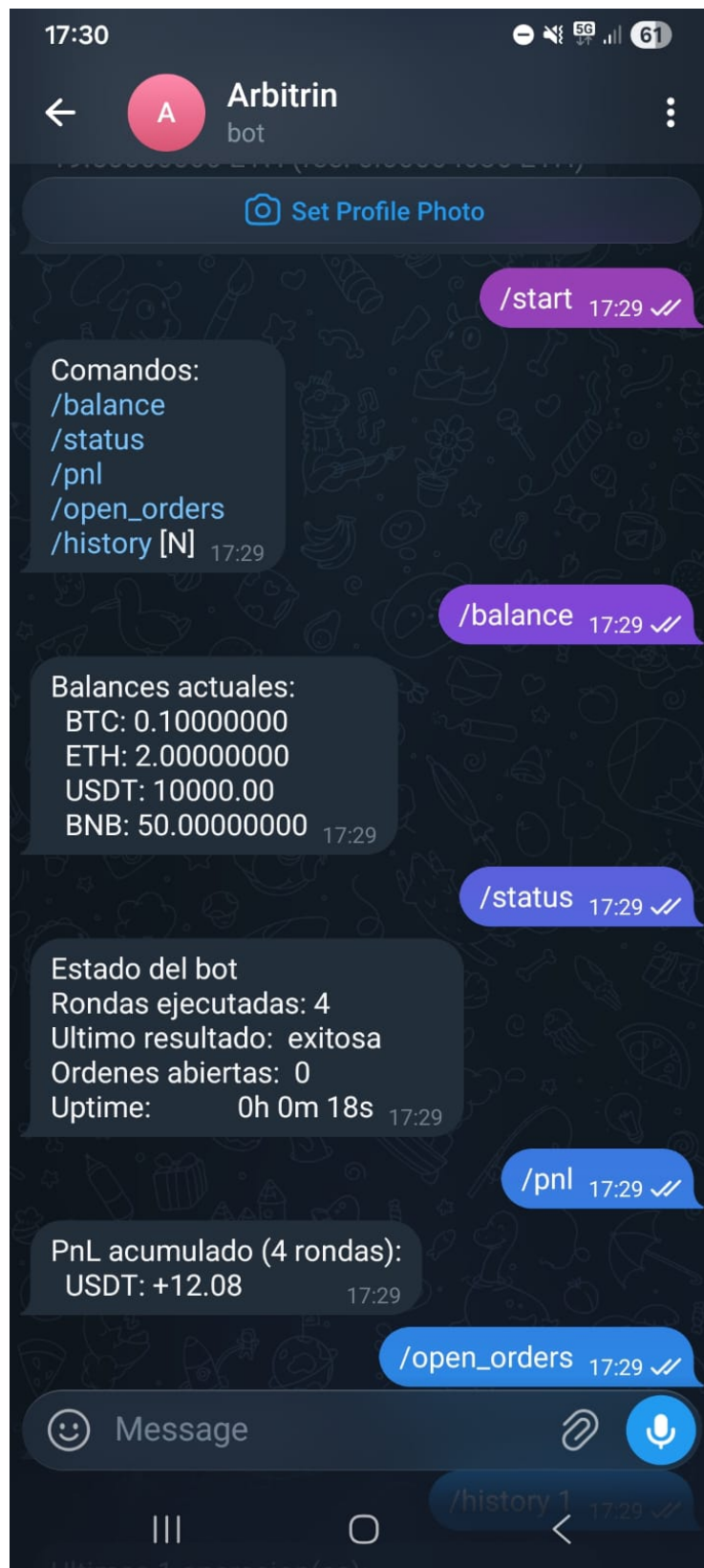


Figura 2: Respuesta del bot a los comandos /start /balance /status /pnl consultando metricas en tiempo real.

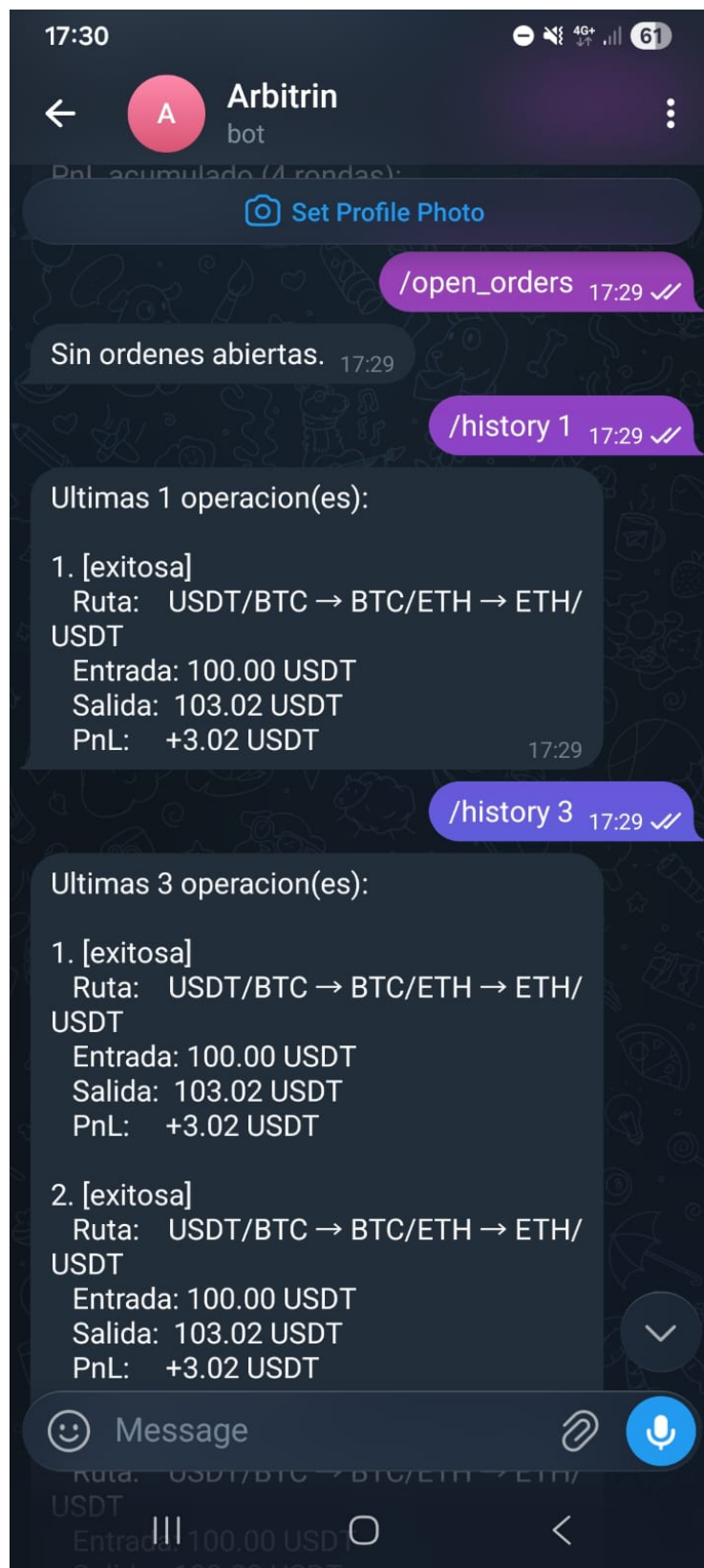


Figura 3: Respuesta del bot al comando /open_orders /history



Figura 4: Respuesta del bot al comando `/history 12` con menos operaciones hechas que requeridas.

4. Modelo de dominio

El modelo de dominio está definido principalmente en `Bot.Domain` y `Binance.API.Types`. Cada concepto relevante del problema tiene su propio tipo en Haskell, lo que hace que las firmas de las funciones sean autoexplicativas y que el compilador detecte usos incorrectos en tiempo de compilación.

4.1. Tipos primitivos

Los tipos más básicos representan los bloques de construcción del dominio:

- **Asset**: tipo suma que enumera los activos soportados: `BTC | ETH | USDT | BNB`.
- **Pair**: par de trading compuesto por un activo base y uno quote: `Pair { base :: Asset, quote :: Asset }`. Por ejemplo, `Pair BTC USDT` representa el par BTC/USDT.
- **Price**: newtype sobre `Double` que evita confundir un precio con cualquier otro valor numérico del sistema.
- **CommissionRate**: newtype sobre `Double` que representa la tasa de comisión de un par (por ejemplo, 0,001 para un 0,1 %).
- **AssetQty**: cantidad de un activo concreto: `AssetQty { qtyAsset :: Asset, qtyAmount :: Double }`. Al llevar el activo junto al monto se evita mezclar cantidades de distintas denominaciones.
- **MarketOrderQty**: tipo suma que especifica cómo dimensionar una orden de mercado: `QtyBase Double` cuando la cantidad se expresa en el activo base del par, o `QtyQuote Double` cuando se expresa en el activo quote. Esta distinción es necesaria porque Binance acepta ambas formas y el lado de la operación determina cuál es la apropiada.

4.2. Snapshot de mercado

`MarketSnapshot` es una fotografía inmutable del estado del mercado en un instante dado. Contiene un `Map Pair PairQuote` donde cada entrada agrupa el precio de compra (`bidPrice`), el precio de venta (`askPrice`) y la tasa de comisión (`pairCommission`) para un par determinado.

Este tipo es el principal dato de entrada de las funciones puras de la capa de transformación: toda la lógica de detección de oportunidades opera sobre un snapshot, sin volver a consultar la API.

4.3. Camino triangular y oportunidad

`TriangularPath` agrupa los tres pares que forman un ciclo de arbitraje (`arbPair1`, `arbPair2`, `arbPair3`). La construcción está protegida por el smart constructor `mkTriangularPath`, que verifica que los pares formen efectivamente un ciclo válido: el quote del par i debe coincidir con el base del par $i + 1$, y el quote del último debe cerrar sobre el base del primero.

`ArbOpportunity` representa una oportunidad concreta detectada sobre un camino: guarda el camino, el monto de entrada (`arbAmountIn`) y el monto estimado de salida (`arbAmountOut`), ambos como `AssetQty`. Las funciones auxiliares `arbProfitAbs` y `arbProfitPerc` calculan la ganancia absoluta y porcentual a partir de estos dos campos.

`Decision` es un tipo suma que expresa el resultado de la etapa de decisión: `NoTrade` cuando ningún camino supera el umbral mínimo de rentabilidad, o `DoTrade` `ArbOpportunity` cuando se encontró una oportunidad que vale la pena ejecutar.

4.4. Plan de ejecución y resultado

Una vez tomada la decisión de operar, la oportunidad se convierte en un `ExecutionPlan`: contiene el camino triangular (`planPath`) y tres `OrderStep` concretos (`planStep1`, `planStep2`, `planStep3`). Cada `OrderStep` especifica el par a operar, el lado de la orden (Buy o Sell) y la cantidad como `MarketOrderQty`. La función `executionPlanSteps` descompone el plan en una lista de tres pasos listos para enviar al exchange.

El resultado de ejecutar esas órdenes queda registrado en `RoundResult`, que acumula los `Fill` obtenidos de Binance (precio real de ejecución, cantidad y comisiones cobradas), los montos de entrada y salida reales, dos campos de PnL neto (`roundNetPnlStart :: Double` expresado en el activo de inicio de la ronda, y `roundNetPnlUsdt :: Double` expresado en USDT) y un `RoundStatus` que indica si la ronda completó las tres órdenes (`RoundSuccess`), si falló alguna intermedia dejando el portafolio en un estado parcial (`RoundPartial`), o si falló completamente (`RoundFailed`).

Los campos de PnL merecen una explicación aparte. En una cadena triangular del tipo $\text{USDT} \rightarrow \text{BTC} \rightarrow \text{ETH} \rightarrow \text{USDT}$, las comisiones de cada paso se cobran *en el activo que se recibe*: la primera orden descuenta comisión en BTC, la segunda en ETH, y la tercera en USDT. Un cálculo ingenuo que solo reste el monto final de USDT al inicial captura únicamente la comisión del último paso; las comisiones cobradas en BTC y ETH quedan fuera del cómputo.

`roundNetPnlUsdt` corrige esto: a partir de los deltas de todos los activos involucrados (entrada, salida y comisiones de cada fill), convierte cada delta a USDT

mediante `Bot.Pricing` y suma los resultados. La conversión distingue el signo del delta para usar el precio correcto del book: los deltas positivos (activos que el bot recibe) se valúan al precio bid (lo que el mercado pagaría por ellos), y los deltas negativos (activos que el bot entrega) al precio ask (lo que costaría reponerlos). Esta asimetría bid/ask hace que el PnL USDT refleje el costo real de transacción con mayor precisión que el precio mid.

`roundNetPnlStart` expresa la misma ganancia en la moneda de inicio de la ronda, calculada directamente como el delta neto de ese activo tras los tres fills. Esto es útil cuando el arbitraje no parte de USDT: el operador puede ver la ganancia en la misma unidad en que invirtió.

5. Principios de programación funcional aplicados

5.1. Pureza e inmutabilidad en el núcleo de arbitraje

La lógica central del bot (detectar oportunidades y decidir si operar) está implementada en funciones completamente puras en `Bot.Arbitraje`. Ninguna de ellas realiza I/O, modifica estado global ni produce efectos secundarios: dado el mismo `MarketSnapshot`, retornan siempre el mismo resultado.

```
1 simulatePath
2   :: MarketSnapshot -> TriangularPath -> AssetQty ->
   Maybe AssetQty
3
4 detectOpportunities
5   :: [TriangularPath] -> MarketSnapshot -> AssetQty -> [
   ArbOpportunity]
6
7 makeDecision
8   :: Double -> [ArbOpportunity] -> Decision
```

Listing 2: Funciones puras del núcleo de arbitraje

Esta separación tiene consecuencias concretas: la lógica de negocio puede ser testeada sin conexión a red, sin mocks y sin preparar estado externo. Además, razonar sobre su corrección es más directo porque no hay efectos implícitos que considerar.

5.2. Tipos como documentación: newtypes y ADTs

El sistema hace un uso intensivo de `newtype` para distinguir valores que en otro lenguaje serían todos `Double`:

```
1 newtype Price          = Price          { unPrice
   :: Double }
2 newtype CommissionRate = CommissionRate { unCommissionRate
   :: Double }
3
4 data MarketOrderQty = QtyBase Double | QtyQuote Double
```

Listing 3: Newtypes y ADTs que hacen los tipos explícitos

Gracias a esto, el compilador rechaza pasar una `CommissionRate` donde se espera un `Price`. Las firmas de funciones actúan como documentación ejecutable: `simulateOneStep :: Map Pair PairQuote -> Pair -> AssetQty -> Maybe SimulatedStep` comunica exactamente qué se necesita y qué se produce, sin comentarios adicionales.

Los tipos suma (*ADTs*) cumplen el mismo rol para los conceptos discretos del dominio: `Asset`, `OrderSide` y `RoundStatus` modelan conjuntos cerrados de valores, y el compilador advierte si algún caso queda sin cubrir en un `case`.

5.3. Abstracción con typeclasses

La interacción con el exchange está definida como una typeclass:

```

1  class Exchange e where
2      checkConnectivity
3          :: MonadIO m
4          => e -> m (Either ExchangeError Bool)
5
6      fetchMarketSnapshot
7          :: MonadIO m
8          => e -> [Asset] -> m (Either ExchangeError
9                               MarketSnapshot)
10
11     fetchBalances
12         :: MonadIO m
13         => e -> m (Either ExchangeError (Map Asset Double))
14
15     executeOrder
16         :: MonadIO m
17         => e -> OrderStep -> m (Either ExchangeError Fill)

```

Listing 4: Typeclass Exchange en Exchange.Interface

El resto del sistema opera contra esta interfaz, sin conocer la implementación concreta. `Bot.Runtime` y `Main` nunca importan `Binance.API.Client` directamente: trabajan con cualquier valor que satisfaga `Exchange e`. Esto desacopla la lógica de negocio del exchange específico y facilita reemplazar Binance por otro exchange, o por una implementación de prueba, sin tocar el núcleo del bot.

5.4. Manejo de efectos con monad transformers

La ejecución de una ronda de trading requiere tres tipos de efecto simultáneos: acceder a configuración de solo lectura, acumular estado mutable y propagar errores. En lugar de combinar estas necesidades ad hoc, el sistema define una mónada dedicada que apila tres transformadores:

```

1  newtype BotM e a = BotM
2      { unBotM :: ReaderT (Env e) (StateT BotState (ExceptT
3                               BotError IO)) a

```

```

3   }
4   deriving newtype
5   ( Functor, Applicative, Monad
6     , MonadReader (Env e)
7     , MonadState   BotState
8     , MonadError   BotError
9     , MonadIO
10  )

```

Listing 5: Definición de BotM en Bot.Runtime

El tipo `Env` que lleva el `ReaderT` agrupa todo lo que es inmutable durante la vida del bot:

```

1  data Env e = Env
2    { envConfig      :: Config          -- parametros de
3      , envExchange  :: e              -- instancia del exchange
4      , envStateRef  :: IORef BotState -- estado compartido con
5      , envStartTime :: UTCTime        -- timestamp de arranque
6      , envUptime    :: UTCTime        -- (para uptime)
7    }

```

Listing 6: `Env` en Bot.Runtime: entorno inmutable del bot

El campo `envStateRef` merece atención especial: es un `IORef BotState` que actúa como canal de comunicación entre el hilo principal del bot y el hilo del listener de comandos. Al finalizar cada ronda, `runOneRound` escribe el nuevo estado en el `IORef`; el listener lo lee cuando recibe un comando de Telegram.

Cada capa del stack tiene una responsabilidad precisa:

- `ReaderT Env`: inyecta la configuración, la instancia del exchange, el `IORef` de estado compartido y el timestamp de inicio sin pasarlos explícitamente en cada llamada.
- `StateT BotState`: acumula el historial de rondas ejecutadas de forma mutable pero controlada.
- `ExceptT BotError`: permite cortocircuitar la ejecución ante un error sin usar excepciones de runtime.
- `IO`: base del stack, habilita las operaciones de entrada y salida necesarias para comunicarse con la API.

5.5. Manejo de errores sin excepciones

Los errores esperables del sistema (fallo de red, orden rechazada, configuración inválida) se representan con tipos explícitos y se propagan mediante `Either`, nunca con excepciones de runtime:

```
1 data ExchangeError
2   = ExchangeConnError Text
3   | ExchangeFetchError Text
4   | ExchangeOrderError Text
5
6 data BotError
7   = BotExchangeError Text
8   | BotExecutionError Text
9   | BotConfigError Text
```

Listing 7: Tipos de error del sistema

Toda función que puede fallar devuelve `Either ExchangeError a` o ejecuta en `BotM` que lleva `ExceptT BotError` en su stack. El compilador obliga a tratar el caso de error en cada sitio de llamada, haciendo imposible ignorarlo silenciosamente. El caso especial de `RoundPartial` en `RoundStatus` ilustra además el manejo de fallos parciales: si la segunda o tercera orden falla después de que la primera ya fue ejecutada (operación irreversible), el sistema registra el estado real en lugar de fingir que todo anduvo bien.

5.6. Representación de texto con `Data.Text`

En Haskell, el tipo estándar `String` es un alias de `[Char]`: una lista enlazada de caracteres. Cada carácter ocupa un nodo separado en el heap, lo que implica un *overhead* de memoria considerable y un costo $O(n)$ para operaciones básicas como concatenación o longitud.

El sistema utiliza `Data.Text` (del paquete `text`) en lugar de `String` para toda representación textual: mensajes de error, campos de configuración, parámetros de las llamadas HTTP y texto formateado para Telegram. `Text` es un arreglo compacto de unidades UTF-16, significativamente más eficiente tanto en memoria como en velocidad de acceso y concatenación.

```
1 -- String: lista enlazada, un nodo por caracter
2 "hola" :: String -- = ['h','o','l','a']
3
4 -- Text: arreglo compacto UTF-16
5 "hola" :: Text -- representacion contigua en memoria
```

Listing 8: String vs Text: representación interna

Las razones concretas para preferir `Text` en este sistema son tres:

1. **Compatibilidad con las bibliotecas de infraestructura.** Las bibliotecas `req` (HTTP) y `aeson` (JSON) trabajan nativamente con `Text`. Los campos de las respuestas de la API de Binance y de Telegram se deserializan directamente como `Text`; usar `String` obligaría a conversiones constantes con `T.pack/T.unpack` en cada frontera.
2. **Coherencia del sistema de tipos.** Al tipar los campos de configuración (`cfgApiKey`, `cfgBaseUrl`, `cfgTelegramToken`, etc.) y los constructores de error (`ExchangeConnError`, `BotExchangeError`, etc.) directamente como `Text`, el compilador garantiza que no se mezclen accidentalmente cadenas de distintos tipos, y los mensajes de error viajan sin conversión desde su punto de origen hasta Telegram.
3. **Eficiencia en el formateo de notificaciones.** Las funciones de `Notification.Telegram` y `Notification.TelegramCommands` construyen mensajes con concatenación intensiva. Con `Text` y el operador `<>`, estas concatenaciones operan sobre arreglos compactos; con `String` cada `++` recorre la lista completa del operando izquierdo.

La extensión `OverloadedStrings` permite escribir literales de texto directamente como `Text` sin conversión explícita:

```
1 {-# LANGUAGE OverloadedStrings #-}
2
3 -- Sin OverloadedStrings:
4 msg :: Text
5 msg = T.pack "Sin_oportunidades_de_arbitraje_rentables."
6
7 -- Con OverloadedStrings (literal inferido como Text por
8   contexto):
9 msg :: Text
10 msg = "Sin_oportunidades_de_arbitraje_rentables."
```

Listing 9: `OverloadedStrings`: literales `Text` sin conversion

Refactor asistido por IA. La migración de `String` a `Text` abarcó 15 módulos en forma simultánea: tipos de dominio, configuración, cliente HTTP, lógica de arbitraje, capa de notificaciones y punto de entrada. Dado el alcance transversal del cambio, el

refactor fue realizado con asistencia de un agente de inteligencia artificial (Claude Code). Comprender los beneficios de `Data.Text` fue el paso conceptual relevante; una vez tomada esa decisión de diseño, el agente propagó el cambio de tipos, actualizó las firmas de funciones, reemplazó las concatenaciones `++` por `<>` y ajustó los usos de `show` con `T.pack` en todos los módulos afectados, verificando la compilación al finalizar. El tiempo total del refactor fue una fracción del que hubiera demandado hacerlo manualmente, sin introducir errores de tipo que el compilador no detectara.

6. Implementación destacada

Esta sección analiza cuatro fragmentos de código que ilustran, en conjunto, cómo los principios del paradigma funcional se aplican de forma concreta a lo largo de las capas del sistema.

6.1. Loop continuo como recursión en BotM

El ciclo principal del bot está implementado como una función recursiva en cola dentro de BotM, en lugar de un loop imperativo externo con `forever`. Esto aprovecha el monad stack para mantener el estado entre iteraciones sin recargar el archivo de persistencia en cada ronda:

```
1  runOneRound :: BotM AppExchange ()
2  runOneRound = do
3      checkConnectivityOrThrow
4      liftIO $ putStrLn "Conectividad OK"
5      snapshot <- fetchMarketSnapshotOrThrow tradingAssets
6      config <- asks envConfig
7      exchange <- asks envExchange
8      stateRef <- asks envStateRef
9      startTime <- asks envStartTime
10     curSt <- get
11     newSt <- liftIO $ handleSnapshot config exchange
        snapshot curSt
12                                     stateRef startTime
13     liftIO $ saveState (cfgStateFile config) (fromBotState
        newSt)
14     liftIO $ writeIORef stateRef newSt
15     put newSt
16
17 botLoop :: BotM AppExchange ()
18 botLoop = do
19     runOneRound `catchError` \err ->
20         liftIO $ putStrLn $ "Error crítico (continuando): "
                ++ show err
21     config <- asks envConfig
22     liftIO $ threadDelay (cfgPollInterval config * 1000000)
23     botLoop
```

Listing 10: Main: botLoop y runOneRound en BotM

Hay tres aspectos que destacan la orientación funcional de este diseño:

- **Estado persistente entre iteraciones via StateT.** `curSt <- get` obtiene el estado acumulado de todas las rondas anteriores directamente del `StateT BotState` del stack. No es necesario recargar el archivo JSON en cada ciclo: la persistencia a disco existe solo para recuperación ante caídas, mientras que el estado en memoria es la fuente de verdad durante la ejecución normal.
- **Aislamiento de errores con `catchError`.** Si `runOneRound` lanza un `BotError` (por ejemplo, fallo de conectividad), `catchError` lo captura, lo registra y permite que el loop continúe en la siguiente iteración. El estado del `StateT` en el punto de la excepción se preserva gracias a la semántica de `StateT` sobre `ExceptT`: el handler recibe el estado anterior al fallo, no un estado parcialmente modificado.
- **Recursión de cola en IO.** La llamada final `botLoop` es una recursión de cola en el contexto de IO: no acumula stack frames entre iteraciones. El loop corre indefinidamente sin crecimiento de memoria por la recursión en sí.

6.2. Simulación de un camino con la mónada `Maybe`

`simulatePath` calcula el resultado estimado de recorrer un camino triangular completo. Su implementación usa *do-notation* sobre la mónada `Maybe` para encadenar tres operaciones que pueden fallar:

```

1  simulatePath
2      :: MarketSnapshot -> TriangularPath -> AssetQty ->
        Maybe AssetQty
3  simulatePath snapshot path amountIn = do
4      s1 <- simulateOneStep quotes (arbPair1 path) amountIn
5      s2 <- simulateOneStep quotes (arbPair2 path) (
6          simulatedAmountOut s1)
7      s3 <- simulateOneStep quotes (arbPair3 path) (
8          simulatedAmountOut s2)
9      return (simulatedAmountOut s3)
10 where
11     quotes = snapshotQuotes snapshot

```

Listing 11: Bot.Arbitraje: `simulatePath`

Cada `<-` dentro del bloque `do` es, en realidad, un `»=` sobre `Maybe`: si `simulateOneStep` devuelve `Nothing` para cualquiera de los tres pasos (porque el par no está disponible en el snapshot en ninguna de sus dos orientaciones posibles), la computación entera retorna `Nothing` sin necesidad de ningún `if` ni manejo explícito del caso de fallo. La función `simulateOneStep` encapsula tanto la búsqueda en el snapshot como el

cálculo del monto de salida según la dirección del par (venta al bid o compra al ask), y devuelve un `SimulatedStep` del que `simulatePath` extrae el monto resultante para pasarlo como entrada al paso siguiente. La secuencia queda expresada de forma lineal y legible, ocultando el plumbing del propagado de errores.

6.3. Detección de oportunidades con list comprehensions

`detectOpportunities` enumera todos los caminos rentables a partir de un snapshot de mercado:

```
1  detectOpportunities
2      :: [TriangularPath] -> MarketSnapshot -> AssetQty -> [
3      ArbOpportunity]
3  detectOpportunities paths snapshot amountIn =
4      [ ArbOpportunity path cappedIn amountOut
5      | path <- paths
6      , let cappedIn = capToLiquidity amountIn path
7      , Just amountOut <- [simulatePath snapshot path
8      cappedIn]
9      , qtyAsset amountOut == qtyAsset amountIn
10     , qtyAmount amountOut > qtyAmount cappedIn
11     ]
11  where
12     quotes = snapshotQuotes snapshot
13     capToLiquidity (AssetQty a q) path =
14         let liqMax = fromMaybe q (maxAmountForPath quotes
15         path (AssetQty a q))
16         in AssetQty a (min q liqMax)
```

Listing 12: Bot.Arbitraje: detectOpportunities

La *list comprehension* integra cuatro responsabilidades en una sola expresión declarativa:

1. Iterar sobre todos los caminos posibles (`path <- paths`).
2. Acotar el monto de entrada a la liquidez disponible en el book (`cappedIn = capToLiquidity amountIn path`): evita simular cantidades que el mercado no puede absorber.
3. Descartar silenciosamente los caminos para los cuales la simulación devuelve `Nothing`, mediante el patrón `Just amountOut <- [...]`.

4. Filtrar los caminos que no son rentables con las dos guardas finales: el activo de salida debe ser el mismo que el de entrada, y la cantidad de salida debe superar al monto acotado de entrada.

El resultado es una lista de oportunidades concretas, ya filtradas y listas para pasarse a `makeDecision`.

6.4. Ejecución de una ronda en BotM

`executeRound` es el punto donde las capas pura y de efectos se unen. Opera dentro de `BotM`, lo que le da acceso al entorno, al estado y al manejo de errores sin recibirlos como parámetros explícitos:

```
1  executeRound :: Exchange e => MarketSnapshot ->
    ExecutionPlan -> BotM e RoundResult
2  executeRound snapshot plan = do
3      env <- ask
4      let steps = executionPlanSteps plan
5          amtIn = extractStepInputAmount (head steps)
6      modify $ \s -> s { bsOpenOrders = steps }
7      (fills, errs) <- executeStepsSequentially (envExchange
          env) steps []
8      result <- case buildRoundResult snapshot amtIn fills
          errs of
9          Left err -> throwError (BotExecutionError
10              ("No se pudo calcular PnL de la ronda: "
11              " <> err))
11          Right rr -> return rr
12      modify $ updateStateWithRound result
13      return result
```

Listing 13: Bot.Runtime: `executeRound`

Hay cuatro aspectos destacables:

- **Inyección de dependencias via `ask`.** La instancia del exchange se obtiene del entorno con `ask`, sin necesidad de pasarla como parámetro. Esto es el patrón *Reader* aplicado directamente.
- **El snapshot como parámetro explícito.** `executeRound` recibe el `MarketSnapshot` del ciclo actual para que `buildRoundResult` pueda convertir los deltas de activos intermedios a USDT usando cotizaciones bid/ask del snapshot (bid para flujos positivos, ask para negativos). El snapshot no se vuelve a consultar

durante la ejecución: se usa el mismo que motivó la decisión de operar, lo que garantiza consistencia entre la simulación y el cómputo del PnL real.

- **PnL como operación que puede fallar.** `buildRoundResult` devuelve `Either String RoundResult`: puede fallar si algún activo involucrado no tiene cotización a USDT en el snapshot (error propagado por `Bot.Pricing`). En ese caso `throwError` propaga un `BotExecutionError` que el loop principal captura con `catchError`, registra por consola y descarta la ronda sin detener el bot.
- **Tolerancia a fallos parciales.** `executeStepsSequentially` ejecuta las órdenes una por una y se detiene ante el primer error, pero no lanza ninguna excepción: retorna los fills obtenidos hasta ese momento junto con la descripción del error. Como las órdenes de mercado son irreversibles, registrar el estado real en `RoundPartial` es la única respuesta correcta.

7. Resultados

Esta sección presenta tres corridas del bot con distintas configuraciones: la primera contra la API real de Binance Testnet, la segunda sobre el **FakeExchange** para ilustrar un ciclo completo con ejecución de órdenes, y la tercera muestra el loop continuo y la interacción con los comandos de Telegram.

7.1. Corrida sobre Binance Testnet

La primera corrida se ejecutó con `BOT_EXCHANGE=binance` apuntando a `testnet.binance.vision`, con un umbral mínimo de ganancia de 0,5% y un capital máximo de 100 USDT.

Análisis de la salida.

- **Conectividad OK.** El bot realizó un `ping` exitoso al endpoint de la API antes de consumir ninguna llamada autenticada. Si este paso falla, el bot termina con un `BotExchangeError` sin intentar obtener precios ni ejecutar órdenes.
- **Comisión de cuenta: 0,0.** La tasa de comisión se obtuvo directamente de la cuenta en la testnet mediante `getAccountInfo`. Binance Testnet asigna comisión cero a las cuentas de prueba; el valor real en producción es de 0,1% (0,001) por operación, que es el valor de respaldo configurado en `.env`.
- **Sin oportunidades de arbitraje rentables.** El bot evaluó los 8 caminos triangulares posibles con los activos BTC, ETH, BNB y USDT. Ninguno superó el umbral mínimo de 0,5% de ganancia neta. Este resultado es el esperado en un mercado eficiente: los precios están arbitrados y la diferencia entre el valor simulado de cada cadena y el capital de entrada no supera los costos de transacción. El tipo de decisión devuelto fue `NoTrade`.

La ausencia de oportunidades no indica un fallo del sistema; es el comportamiento correcto. El bot cumplió el ciclo completo (conectividad, snapshot, detección, decisión) y concluyó sin ejecutar órdenes, exactamente como se esperaba.

7.2. Corrida con exchange simulado

Para demostrar un ciclo completo de arbitraje con ejecución de órdenes, se utilizó `BOT_EXCHANGE=fake`. El **FakeExchange** opera con precios fijos configurados intencionalmente para crear una oportunidad rentable, sin realizar ninguna llamada de red.

Los precios de referencia usados por el **FakeExchange** son los siguientes:

Par	Bid	Ask
USDT/BTC	0,0000208	0,0000212
BTC/ETH	19,5	20,5
ETH/USDT	2 550	2 650

Análisis de la salida.

- **Oportunidad detectada.** La función pura `detectOpportunities` identificó el camino `USDT → BTC → ETH → USDT` como rentable. La simulación estimó una salida de 103,12 USDT sobre 100 USDT de entrada, lo que representa una ganancia bruta estimada del 3,12%, por encima del umbral mínimo del 0,5%. `makeDecision` devolvió `DoTrade opp`.
- **Ejecución de las tres órdenes.** El `ExecutionPlan` generado a partir de la oportunidad fue validado, cuantizado a la precisión del activo y enviado a `executeRound`. Las tres órdenes de mercado se ejecutaron secuencialmente:
 1. Venta de USDT por BTC al precio bid de USDT/BTC.
 2. Venta de BTC por ETH al precio bid de BTC/ETH.
 3. Venta de ETH por USDT al precio bid de ETH/USDT.

El estado durante la ejecución se registró en `bsOpenOrders`; al completarse, `updateStateWithRound` actualizó el `BotState` con los resultados reales.

- **RoundSuccess y PnL real vs. simulado.** El estado final fue `RoundSuccess`: las tres órdenes completaron sin error. El PnL real fue 3,02 USDT, ligeramente menor que la ganancia simulada de 3,12 USDT. La diferencia de 0,10 USDT corresponde al *slippage* entre el precio utilizado en la simulación y el precio real de los fills sintéticos generados por el `FakeExchange`, que aplica una pequeña variación aleatoria para simular condiciones reales de mercado.
- **Estado del bot y persistencia.** El resumen final muestra el `BotState` tras la ronda: 1 ronda ejecutada, 0 errores, PnL acumulado de 3,02 USDT. Los balances delta reflejan los movimientos netos de activos: residuos mínimos de BTC y ETH producto del redondeo de las cantidades, y una ganancia neta de 2,92 USDT en USDT (PnL bruto menos comisiones deducidas). Al finalizar el ciclo, `Bot.Persist` serializó este estado en `bot_state.json`, garantizando que el PnL acumulado persista entre sesiones.

7.3. Loop continuo y comandos de Telegram

Esta corrida muestra el bot ejecutando múltiples ciclos en forma continua con `BOT_EXCHANGE=fake` y `BOT_POLL_INTERVAL=10`, y la interacción con los comandos de Telegram en tiempo real.

Las Figuras 2, 3 y 4 de la Sección 3.6 muestran las respuestas a los comandos de consulta enviados durante la ejecución del loop. El listener de comandos corre en paralelo al ciclo principal y responde sin interrumpirlo.



Figura 5: Mensaje de Telegram enviado por el bot al detectar una oportunidad de arbitraje.

8. Conclusiones

8.1. Aprendizajes

El desarrollo de este trabajo puso en práctica los principios centrales del paradigma funcional en un dominio concreto y con restricciones reales. Los aprendizajes más significativos del proceso fueron:

- **La separación entre código puro y efectos no es solo estética.** Definir el núcleo de arbitraje como funciones puras tuvo consecuencias tangibles: la lógica de detección de oportunidades puede ser razonada, probada y modificada de forma completamente independiente de la API de Binance.
- **El sistema de tipos actúa como primera línea de defensa.** El uso de newtypes (`Price`, `CommissionRate`) y tipos suma (`MarketOrderQty`, `OrderSide`) eliminó una categoría entera de bugs (mezcla de unidades y de direcciones de orden) que en un lenguaje con tipos más permisivos solo aparecería en tiempo de ejecución.
- **Los monad transformers hacen explícito lo que otros lenguajes ocultan.** La firma `ReaderT Env (StateT BotState (ExceptT BotError IO))` es más verbosa que un objeto con campos mutables, pero hace visible exactamente qué efectos son posibles en cada parte del código.
- **Las comisiones cambian cualitativamente el problema.** Ignorar las comisiones convierte la detección en un ejercicio trivial; incorporarlas con tasas reales obtenidas de la API revela que la mayoría de los caminos que parecen rentables en bruto dejan de serlo una vez aplicados los costos reales de las tres órdenes.

8.2. Limitaciones

El diseño actual presenta las siguientes limitaciones:

- **Órdenes de mercado y slippage.** El uso exclusivo de órdenes de mercado garantiza la ejecución, pero no el precio. Entre el momento en que se simula el camino y el momento en que se ejecuta la primera orden, los precios del book pueden haber cambiado, haciendo que el resultado real difiera del estimado.
- **Universo de activos fijo.** Los cuatro activos soportados (BTC, ETH, BNB, USDT) están determinados al momento de la llamada a `allTriangularPaths`. Ampliar el universo requiere modificar el código.

- **Evaluación secuencial de caminos.** Los caminos triangulares se simulan uno por uno. Con un universo de activos mayor, esto podría convertirse en un cuello de botella.

8.3. Posibles extensiones

A partir del diseño actual, las extensiones más naturales serían:

- **Soporte para múltiples exchanges.** La typeclass `Exchange` ya prevé esta extensión: bastaría con implementar una nueva instancia para otro exchange sin modificar la lógica de arbitraje.
- **Backoff exponencial en fallos.** Ante errores consecutivos de conectividad, duplicar el intervalo de espera hasta un máximo configurable, evitando saturar el exchange durante una interrupción de servicio.
- **Historial acotado de rondas.** El campo `bsRounds` del `BotState` crece de forma ilimitada en memoria. Limitarlo a las últimas N rondas evitaría el crecimiento sin tope en runs de larga duración.